
Laboratorio No. 1 Semana #13 – APIS con swagger & postman

Nombre: Diego Licardié **Carné:** 1251825

Objetivo: Comprender y consumir una API REST mediante Swagger UI y Postman, identificando endpoints, métodos HTTP, parámetros, cuerpos JSON, códigos de respuesta y operaciones CRUD. El estudiante deberá interpretar la documentación de una API, ejecutar solicitudes y resolver un reto práctico utilizando evidencia técnica.

Investigación previa:

1. ¿Qué es una API REST?
2. ¿Qué es un endpoint?
3. ¿Qué diferencia existe entre una URL y un endpoint?
4. ¿Qué función cumplen los métodos HTTP GET, POST, PUT/PATCH y DELETE?
5. ¿Qué es JSON y por qué es común en las APIs?
6. ¿Qué es un código de estado HTTP? Mencione ejemplos de 200, 404 y 500.
7. ¿Qué información se puede encontrar en la respuesta de una API?
8. ¿Qué ventajas tiene utilizar Postman para probar una API antes de programarla?
9. ¿Qué es y para qué sirve Swagger UI?
10. ¿Qué función cumple Postman en el desarrollo y prueba de APIs?

1. Preparación del entorno

Instalar o utilizar Postman: <https://www.postman.com/downloads/>

API a explorar, Swagger Petstore 3: <https://petstore3.swagger.io/>

Importante: La actividad se realizará mediante solicitudes HTTP GET y análisis de las respuestas JSON. No se requiere API Key para esta práctica.

2. Exploración inicial

Ingresa a Swagger Petstore y localice las operaciones relacionadas con /pet

- Identifique un endpoint que permita consultar una mascota por ID.
GET /pet/{petId}
- Indique el método HTTP utilizado.
Get
- Indique qué parámetro necesita la solicitud.
Un Id
- Ejecute la solicitud con Try it out y registre el código HTTP recibido.
200
- Explique con sus palabras qué información contiene la respuesta.
esta contiene un JSON con el Id que solocite, el nombre de la mascota y una foto.

Evidencia: captura de Swagger mostrando la operación y su respuesta.

Name	Description
petId * required integer(\$int64) (path)	ID of pet to return

Curl

```
curl -X 'GET' \
  'https://petstore3.swagger.io/api/v3/pet/10' \
  -H 'accept: application/json'
```

Request URL

```
https://petstore3.swagger.io/api/v3/pet/10
```

Server response

Code	Details
200	Response body <pre>{ "id": 10, "name": "Rufus", "photoUrls": ["example.com"], "tags": [] }</pre> Response headers <pre>access-control-allow-headers: Content-Type,api_key,Authorization access-control-allow-methods: GET,POST,DELETE,PUT access-control-allow-origin: * access-control-expose-headers: Content-Disposition content-length: 62 content-type: application/json date: Fri,02 Oct 2026 20:28:51 GMT server: Jetty(9.4.53.v20231009)</pre>

3. Crear un recurso con POST

Utilice POST /pet para crear una mascota. Construya un JSON propio; no copie exactamente el ejemplo de Swagger.

El JSON debe incluir, como mínimo:

id
name
status
category
tags

Registre el ID utilizado y verifique la respuesta. Luego consulte nuevamente la mascota por medio de GET /pet/{petId}.

Evidencia: JSON enviado + respuesta de creación + consulta posterior.

Edit Value Schema

```
{
  "id": 1251825,
  "name": "Chuchin",
  "category": {
    "id": 1,
    "name": "Dogs"
  },
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 0,
      "name": "string"
    }
  ],
  "status": "available"
}
```

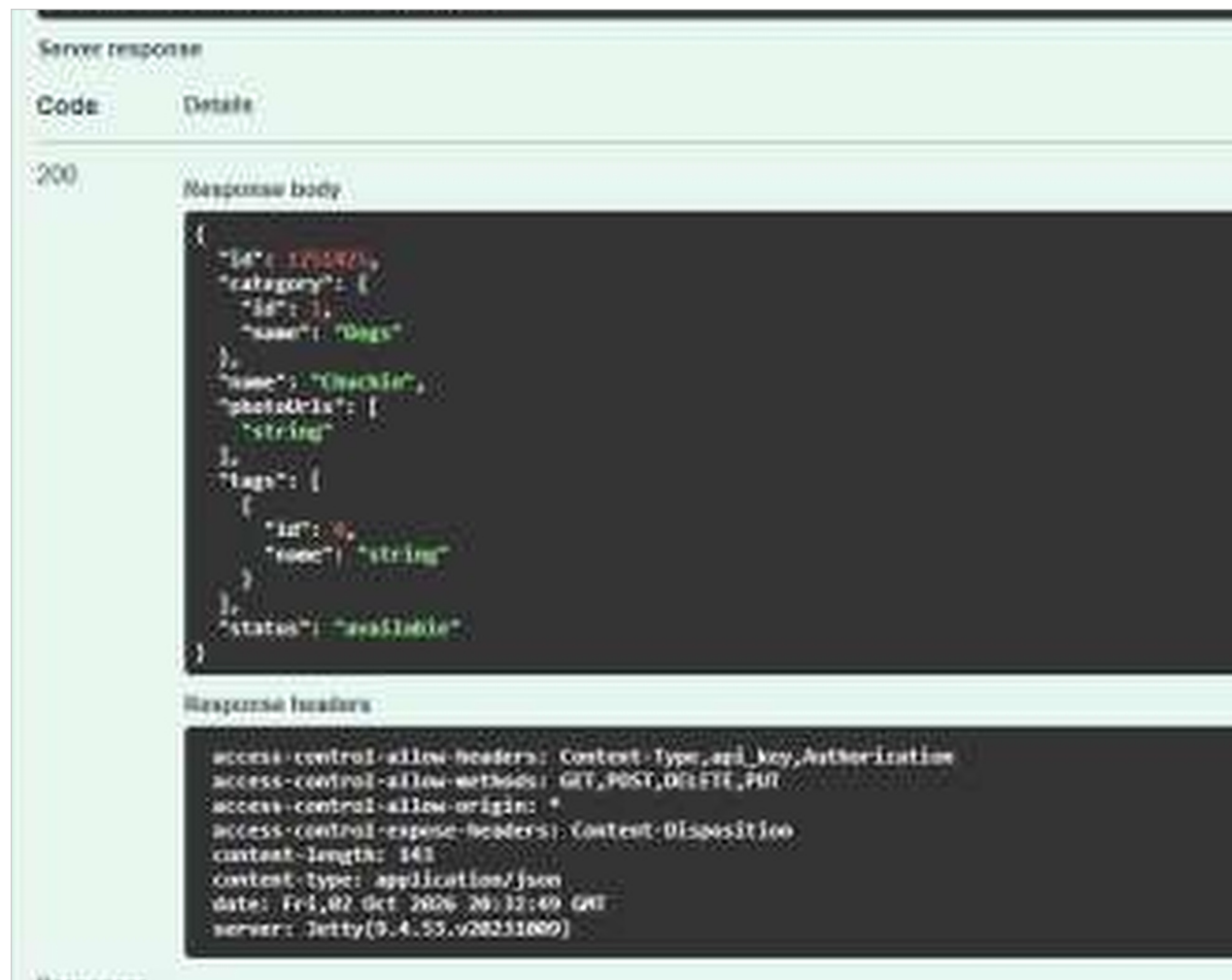
Responses

Curl

```
curl -X 'POST' \
  'https://petstore3.swagger.io/api/v3/pet' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "id": 1251825,
    "name": "Chuchin",
    "category": {
      "id": 1,
      "name": "Dogs"
    },
    "photoUrls": [
      "string"
    ],
    "tags": [
      {
        "id": 0,
        "name": "string"
      }
    ],
    "status": "available"
  }'
```

Request URL:

https://petstore3.swagger.io/api/v3/pet



Server response

Code	Details
200	Response body <pre>{ "id": 1705421, "category": { "id": 1, "name": "Dogs" }, "name": "Chachio", "photoUrls": ["string"], "tags": [{ "id": 0, "name": "string" }], "status": "available" }</pre> Response headers <pre>access-control-allow-headers: Content-Type,api_key,Authorization access-control-allow-methods: GET,POST,DELETE,PUT access-control-allow-origin: * access-control-expose-headers: Content-Disposition content-length: 141 content-type: application/json date: Fri, 02 Oct 2026 20:32:49 GMT server: Jetty(9.4.53.v20231009)</pre>

4. Modificar el recurso

- Modifique la mascota creada utilizando PUT /pet. Cambie al menos dos atributos, por ejemplo nombre y estado.
- ¿Qué método HTTP utilizó y por qué?
put /pet
- ¿Qué datos fueron enviados en el body?
cambiados nombre y el status
- ¿Qué código HTTP devolvió la API?
- 200
- ¿Cómo comprobó que el cambio realmente ocurrió?



- Usando el método GET

Request body

Update an existent pet in the store

Edit With Schema

```
{
  "id": 1251825,
  "name": "Juan",
  "category": {
    "id": 1,
    "name": "Dogs"
  },
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 0,
      "name": "string"
    }
  ],
  "status": "unavailable"
}
```

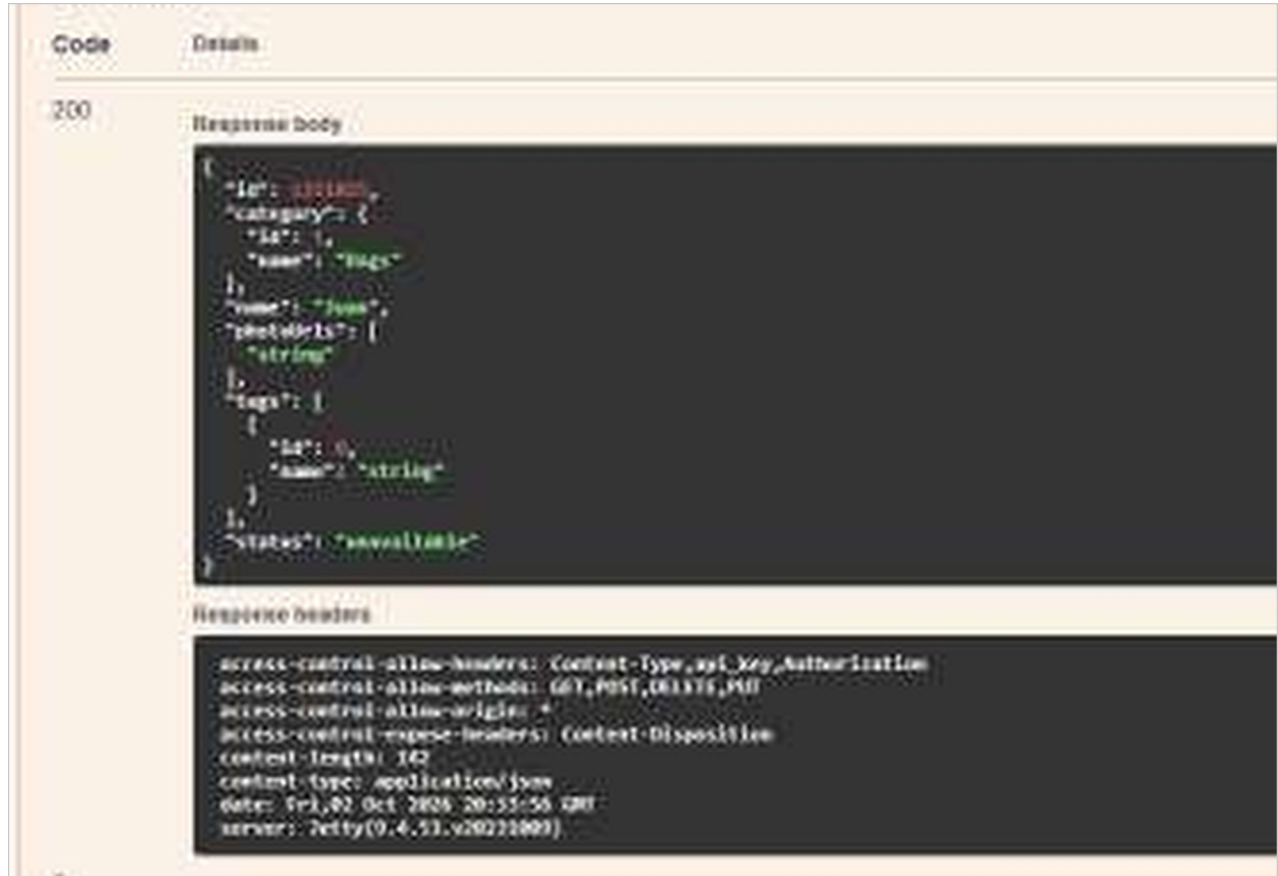
Curl

```
curl -X 'PUT' \
  https://petstore.swagger.io/api/v3/pet \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "id": 1251825,
    "name": "Juan",
    "category": {
      "id": 1,
      "name": "Dogs"
    },
    "photoUrls": [
      "string"
    ],
    "tags": [
      {
        "id": 0,
        "name": "string"
      }
    ],
    "status": "unavailable"
  }'
```

Request URL

https://petstore.swagger.io/api/v3/pet

Secret response



```
Code  Details
200
Response Body
{
  "id": 123456,
  "category": {
    "id": 1,
    "name": "Dogs"
  },
  "name": "Fiona",
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 0,
      "name": "string"
    }
  ],
  "status": "available"
}

Response Headers
access-control-allow-headers: Content-Type, xsl, Xeg, Authorization
access-control-allow-methods: GET, POST, DELETE, PUT
access-control-allow-origin: *
access-control-expose-headers: Content-Disposition
content-length: 142
content-type: application/json
date: Fri, 02 Oct 2026 20:55:56 GMT
server: Jetty(9.4.53.v20221003)
```

5. Consultar utilizando parámetros

- Utilice GET /pet/findByStatus y consulte las mascotas con estado available.
- Identifique el parámetro utilizado.
status
- Explique dónde se envía ese parámetro.
en la query string
- Observe la estructura de la respuesta.
- Indique qué sucede si cambia el valor del parámetro por pending.

la API devuelve otro conjunto de mascotas, esta vez todas con "status": "pending"

Name	Description
status required	Status values that need to be considered for filter
string (query)	sold

Curl

```
curl -X 'GET' \
  'https://petstore3.swagger.io/api/v3/pet/findByStatus?status=sold' \
  -H 'accept: application/json'
```

Request URL

```
https://petstore3.swagger.io/api/v3/pet/findByStatus?status=sold
```

Server response

Code	Details
200	Response body <pre>[{ "id": 9223372036854000000, "name": "Rex", "photoUrls": ["string"], "tags": [], "status": "sold" }, { "id": 1089322, "category": { "id": 1, "name": "Dogs" }, "name": "Firulais_URL_Modificado", "photoUrls": [], "tags": [{ "id": 1, "name": "MIA" }], "status": "sold" }, { "id": 7246159,</pre>

6. Reproducir las operaciones en Postman

- Ahora reproduzca en Postman las operaciones realizadas en Swagger. Cree una colección llamada 'MIA - Semana 13 - API'.



- La colección debe contener, como mínimo:
- GET – Consultar mascota por ID

The screenshot shows a REST client interface. The top section displays a 'New Request' with the method 'GET' and the URL 'https://petstore.swagger.io/v2/pet/1251825'. Below this, a status bar indicates '200 OK' and '200 OK'.

```
1  {
2    "id": 1251825,
3    "category": {
4      "id": 1,
5      "name": "Dogs"
6    },
7    "name": "Bella",
8    "photoUrls": [
9      "string"
10   ],
11   "tags": [
12     {
13       "id": 0,
14       "name": "string"
15     }
16   ],
17   "status": "available"
18 }
```



- POST – Crear mascota

```
New Request
POST https://petstore.swagger.io/v2/pet

Body
application/json

{
  "id": 123456,
  "name": "fluffy",
  "category": { "id": 1, "name": "Dogs" },
  "photoUrls": [ "https://petstore.swagger.io/v2/pet/123456/photo" ],
  "tags": [ { "id": 1, "name": "available" } ],
  "status": "available"
}
```

200 OK 377ms 100% 100% (Success)

```
{
  "id": 123456,
  "category": {
    "id": 1,
    "name": "Dogs"
  },
  "name": "fluffy",
  "photoUrls": [
    "https://petstore.swagger.io/v2/pet/123456/photo"
  ],
  "tags": [
    {
      "id": 1,
      "name": "available"
    }
  ],
  "status": "available"
}
```



- PUT – Modificar mascota

```
PUT https://www.petfinder.com/v3/put

Data Headers Auth: Bearer Token Headers: X-Auth-Token: 12345678901234567890

none PUT-Data x-www-form-urlencoded raw binary GraphQL

1 {
2   "id": 12345678,
3   "name": "Bismarck",
4   "category": { "id": 1, "name": "Perrero" },
5   "photoUrls": [ "https://images.petfinder.com/12345678.jpg" ],
6   "tags": [ { "id": 1, "name": "Amigable" } ],
7   "status": "available"
8 }
```

200 OK 11 ms 104 B Show Response

```
1 {
2   "id": 12345678,
3   "category": {
4     "id": 1,
5     "name": "Perrero"
6   },
7   "name": "Bismarck",
8   "photoUrls": [
9     "https://images.petfinder.com/12345678.jpg"
10  ],
11  "tags": [
12    {
13      "id": 1,
14      "name": "Amigable"
15    }
16  ],
17  "status": "available"
18 }
```



- GET – Buscar por status



- DELETE – Eliminar mascota



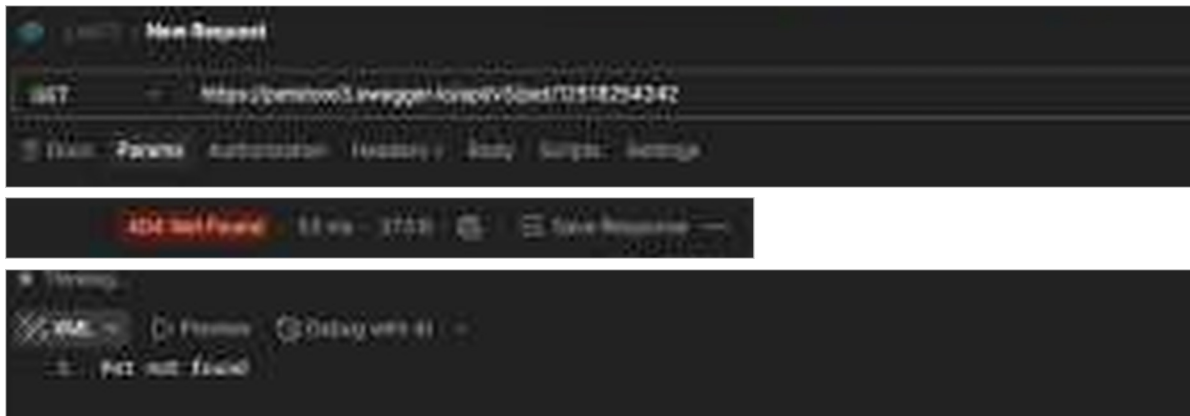
- En cada solicitud configure manualmente método, URL, parámetros, headers y body cuando corresponda. No utilice únicamente la opción de importar una colección.

7. Prueba de errores

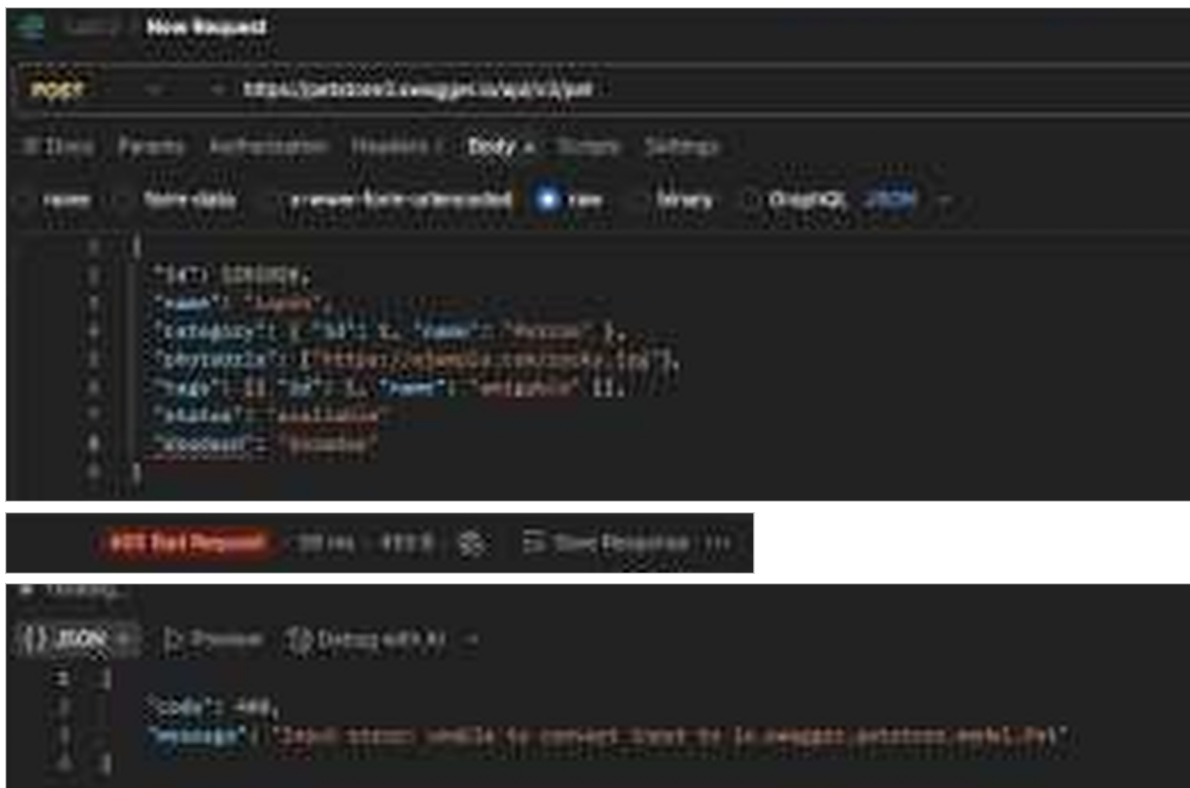
Una API también debe analizarse cuando algo sale mal. Realice al menos tres pruebas que generen respuestas diferentes a una solicitud exitosa.

Ejemplos de escenarios:

1. Consultar un ID inexistente.



2. Enviar un JSON incorrecto o incompleto.



3. Utilizar un parámetro con un valor no esperado.



The screenshot shows the Swagger UI for the Petstore3 API. The endpoint `https://petstore3.swagger.io/api/v3/pet/findByStatus` is selected. The response is a 400 Bad Request error with the following message: "That error: query parameter 'status' value 'available' is not in the allowed values: [available, pending, sold]".

4. Intentar eliminar un recurso que ya no existe.



The screenshot shows the Swagger UI for the Petstore3 API. The endpoint `https://petstore3.swagger.io/api/v3/pet/123456789` is selected. The response is a 400 Bad Request error with the following message: "That error: path id '123456789' is not a valid long integer".

Para cada prueba registre:

- Solicitud realizada.
- Código HTTP recibido.
- Respuesta de la API.
- Qué esperaba que ocurriera.
- Qué ocurrió realmente.
- Explicación de la causa

8. API Detective

Suponga que usted recibe el siguiente requerimiento de un sistema:

"El sistema debe registrar una mascota, consultar su información, actualizar su estado y finalmente eliminarla. Si el recurso no existe, el sistema debe poder identificarlo mediante la respuesta de la API."

Resuelva el reto:

- Defina qué endpoint utilizaría para cada operación.
- Defina el método HTTP correspondiente.
- Indique qué respuesta debe analizar el programa.
- Ejecute el flujo completo en Postman.

Operación	Endpoint	Método	Qué analiza el programa
Registrar	/pet	POST	código 200 y el JSON devuelto con el id
Consultar	/pet/{petId}	GET	código 200 y los campos (name, status)
Actualizar estado	/pet	PUT	código 200 y que status cambió
Eliminar	/pet/{petId}	DELETE	código 200
Detectar que no existe	/pet/{petId}	GET	código 404 : así sabe que el recurso ya no existe

Entrega

Dentro del repositorio de GitHub, cree la carpeta semana-13/lab1/ y coloque en ella un PDF con la solución del laboratorio.

Incluya también los archivos de respaldo generados durante la práctica, así como los pantallazos que evidencien las pruebas realizadas.

Enviar link de github a plataforma de la universidad.